ECE498SM: V2V Lidar Communication in Occluded Environments

Mark Craft
mscraft2@illinois.edu

Kushagra Tiwary
ktiwary2@illinois.edu

Abstract

In a world full of self driving cars, Vehicle-2-Vehicle communication (V2V) will be key to ensuring safety especially while operating in congested and occluded environments. We use Point-Cloud Alignment from different viewpoints to show how Lidar data can be leveraged from different vehicles to find occluded objects and subsequently notify the other car.

1 Introduction

One of the greatest challenges that self driving cars face is navigating through complex intersections due to the high level of uncertainty that is present. One of the challenges is navigating through occluded spaces where the self-driving car must exercise even more caution due to the lack of visibility from different trucks and cars and sharp turns.

We aim to solve this occlusion problem assuming that there exists an environment filled with self-driving cars that can communicate and send packets to each other in almost real-time. To solve the occluded objects problem, we looked at lidar points because of the position accuracy and the precise depth of each of the lidar point cloud points in space.

We take inspiration from the classical image stitching pipeline from Computer Vision and put it to use to stitch lidar point clouds from two different vehicle viewpoints of the same intersection. Through this we want to show that the excess information from both the viewpoints will lead to a better confidence for the existing bounding box detections and even lead to detections of previously occluded objects.

Unfortunately, given the time we could only show that the point cloud alignment stitches two viewpoints accurately of the intersection. We show through qualitative measures that we can see otherwise occluded objects in the the post stitching output. We give two quantitative metrics: analysis of detection confidence in the post and pre-stitched viewpoints and the amount of new good detections between the post and pre-stitched viewpoints as a metric to evaluate our method. We leave this as future work.

2 Related Work

2.1 Image Stitching Pipeline

We first take inspiration from the Image Stitching Pipeline that stitches 2D images together to create panaromas. An important aspect of the problem is to find keypoints. Key point detection is very important as it finds key points in each of the viewpoints to find matches. The matches are then used to find the transformation matrix and warp the two viewpoints together. The keypoint detection is the key and there are a lot of techniques to match in 2D space. These include SIFT (6), DoG, Harris Corner Detector (2) etc. Since we try to do the same thing in 3D space we first try compressing the lidar point cloud in 2D space then 3D-SIFT, DoG and eventually the 3D FeatNet (9) which is a weakly supervised keypoint detection algorithm loosely based on SIFT.

This keypoint detection scheme is used with the classical image stitching pipeline which is described below in the Methodology section.

3 Methodology

3.1 Datasets

There are a couple of famous datasets with LiDar that are made specifically for self driving cars since it includes calibration and vision information. The datasets that we looked at were the following: Ford Campus Lidar Datasets, KITTI (1), Stanford Track Collection.

Due to the inherent constraints of occluded objects that our project presents the stanford dataset were dismissed since there were limited occlusions. The Stanford Track dataset was a car moving on a track therefore there were limited occlusions. Due to the popularity of the KITTI dataset, we decided to analyze that first.

Currently, there are no datasets that provide LiDar point clouds of the same area from the perspective of multiple vehicles. We would want collect this type of data ourselves but due to time constraints, we wanted to test our approach on a modified version of an existing dataset. Thus, to show the efficacy of our approach, we conduct experiments on the KITTI dataset.

To mock our desired data of having LiDar points of the same area from multiple viewpoints, we treat different timesteps in our data as different viewpoints. There are some downsides to this approach; moving objects will be in different positions in different timesteps. Despite this downside, we still found our approach to be effective and particularly in the case where there only static objects in the scene.

3.2 Formulating as Image Stitching

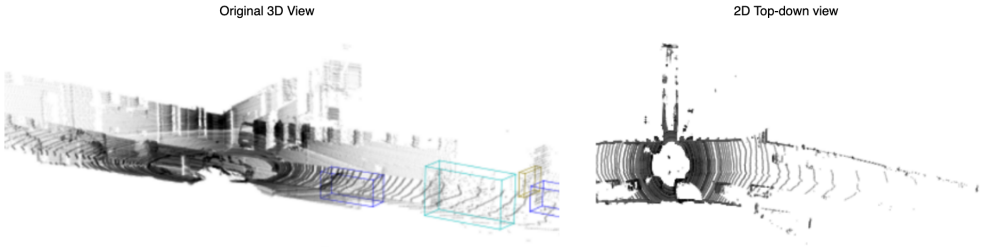

Figure 1: 3D view of a LiDar point cloud and its corresponding 2D top-down projection view.

Prior to attempting to do point-cloud alignment on our LiDar data, we wanted to test how effective it would be to formulate our problem directly as an image stitching problem. Image stitching could be directly applied to the camera views of two separate vehicles. However, this could only be done if the two vehicles could see the same partial scene. Thus, it would only be useful in a subset of scenarios, and we would lose a lot of information that we would get if we used LiDar points.

To do image stitching on LiDar data, we first need a way to project our 3D points into a 2D space. The simplest method we could do this while perserving the most important structures is to look at our 3D point from a top-down view; we would just set the z (height) of all of the LiDar points to 0. After projecting the 3D LiDar points of two different views into a 2D top-down view as seen in Figure 1, we can apply image stitching on it.

The problem of image stitching has 5 main components:

- Detect 2D keypoint features in each image
- Compute distances between two sets of descriptors
- Generate putative matches based on pairwise descriptor matches
- Use RANSAC to find transformation matrix

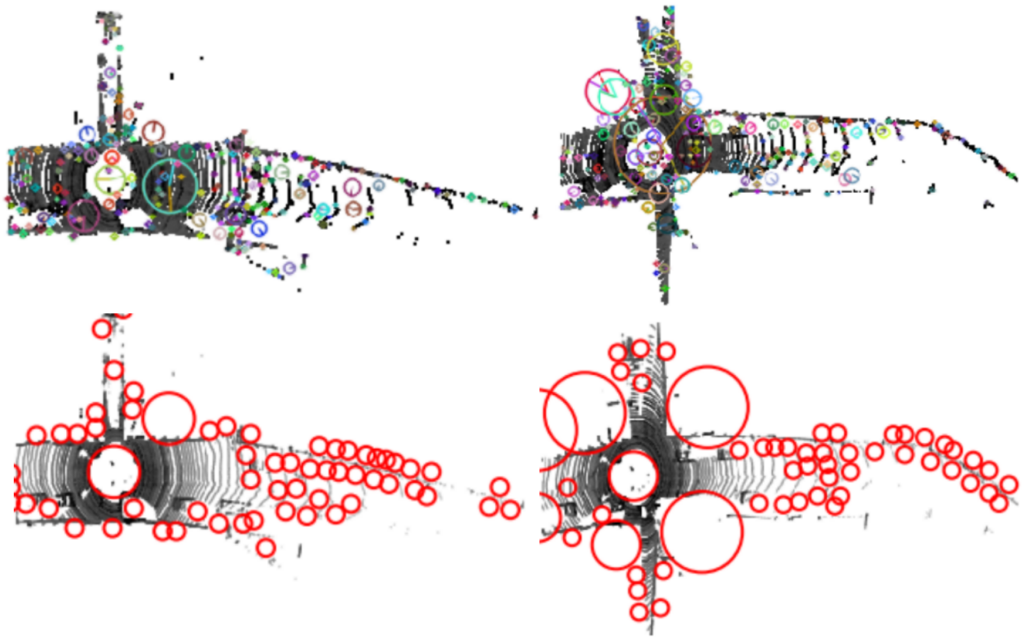

Figure 2: SIFT (top row) and DoG (bottom row) keypoints on the top-down view of two different timesteps of the same road in the KITTI dataset.

- Align images together

One of the most important parts of this algorithm is finding a good keypoint detector to find correspondences between the two images. In the case of image stitching we tried both SIFTand Difference of Gaussians (DoG)keypoint detectors. However as seen in Figure 2, both of the feature descriptors were not able to get particularly useful features. In the case of the DoG method, it attempts to find different sized "blobs" as features. However, in our top-down LiDar view, most of the features that are found are from the spacing in between the LiDar scan and are not keypoints relevant to the scene; there would not be many if at all good correspondences between two different views. Similarly, SIFT also faces problems in detecting keypoint features in the artifacts of the LiDar scan rather than from the scene itself.

Even from looking at the task as a human, it is difficult; it would only be possible to try to align the two images by using the walls detected by the LiDar. However, even by doing it with this method, instances such as a straight lane, or most importantly, four-way intersections would be next to impossible. By projecting our 3D LiDar points into this 2D space, we lose most of the structure that would be needed to align the views together. Thus, we next tried to modify this pipeline to the 3D LiDar points directly.

3.3 Point Cloud Alignment Pipeline

To do point cloud alignment on the 3D LiDar points directly, we apply the image stitching algorithm but modified for 3D points. We have 5 main parts to our pipeline and for each, there are hyperparameters that affect the tradeoff between accuracy and speed for our proposed method.

1. Detect 3D keypoint features on each LiDar view

Unfortunately, unlike for 2D image keypoint detectors, there are not many readily available 3D keypoint detectors. However, by looking at recent literature, we saw that there were two main types of 3D keypoint detectors; handcrafted ones, like the ones commonly used for image stitching, and learned 3D keypoint detectors. The 3 most commonly used handcrafted 3D feature descriptors include: Fast Point Feature Histogram (FPFH) (7), Spin-Image (SI) (4), and Unique Shape Context (USC) (8) among others (11; 3). In addition, the 3 most commonly used learned 3D feature descriptors include:

Compact Geometric Features (CGF) (5), 3DMatch (10), and 3DFeat-Net (9). When comparing all of these 3D feature descriptors, 3DFeat-Net had the best performance.

2. Compute distances between two sets of descriptors

After computing the feature descriptors in two views, we then compute pairwise distances between the two views using euclidean distance. In this step, we only keep the smallest pairwise distances.

3. Generate putative matches based on pairwise descriptor matches

After computing the pairwise distances between our descriptors, we can choose to keep only some putative matches. This is generally done through the use of a threshold on the distance or only keeping the top x number of matches. For our approach, we found keeping all of the matches still worked sufficiently fast yet would produce the most accurate results.

4. Use RANSAC to find transformation matrix

After finding the putative matches based on pairwise descriptor matches, we run random sample consensus (RANSAC) on them. RANSAC generally works by repeating the following steps:

- Select a random subset of the original data. Call this subset the hypothetical inliers.
- A model is fitted to the set of hypothetical inliers.
- All other data are then tested against the fitted model. Those points that fit the estimated model well, according to some model-specific loss function, are considered as part of the consensus set.
- The estimated model is reasonably good if sufficiently many points have been classified as part of the consensus set.
- Afterwards, the model may be improved by re-estimating it using all members of the consensus set.

Furthermore, we used the modified RANSAC algorithm used in SUN3Dto work in 3D space. By using RANSAC, we estimate a 3×4 transformation matrix.

5. Align point-cloud matches to create new view

The transformation matrix estimated through RANSAC is then used to align the two 3D LiDar point clouds together.

4 Results

We show the results of our pipeline on an intersection viewed from two different viewpoints as shown in Figure 5. In this figure, we illustrated both the keypoint matches on the two views as well as their stitched result. As we can see from the figure, the result of the stitching is accurate.

Now that we are able to stitch viewpoints from multiple LiDar point clouds together, we wanted to illustrate how by communicating LiDar point clouds to other vehicles and stitching them together, we can detect objects in occluded environments as seen in Figure 6. In the figure above, we can see that there is some object in the bottom-left of the left view where there is a cluster of red dots (keypoints detected). Additionally, we can we in the same spot in the middle view, we can see part of the left side of an object, but it is ambiguous if there is an object or what it is. In the right stitched view, we can finally see that there was a car being occluded by the car in front of it. Without either of the two views, this car would have not have been detectable. Through the use of this method, we can increase the safety of autonomous vehicles using V2V technology as they would be able to understand more of their surroundings as well as detecting these otherwise occluded objects that could potentially inpair the safety of the vehicle.

5 Conclusions

5.1 Discussion

In this paper, we showed how different viewpoints of a complex intersection can be stitched using Point-Cloud Alignment to further understand the scene. V2V communication provides even greater

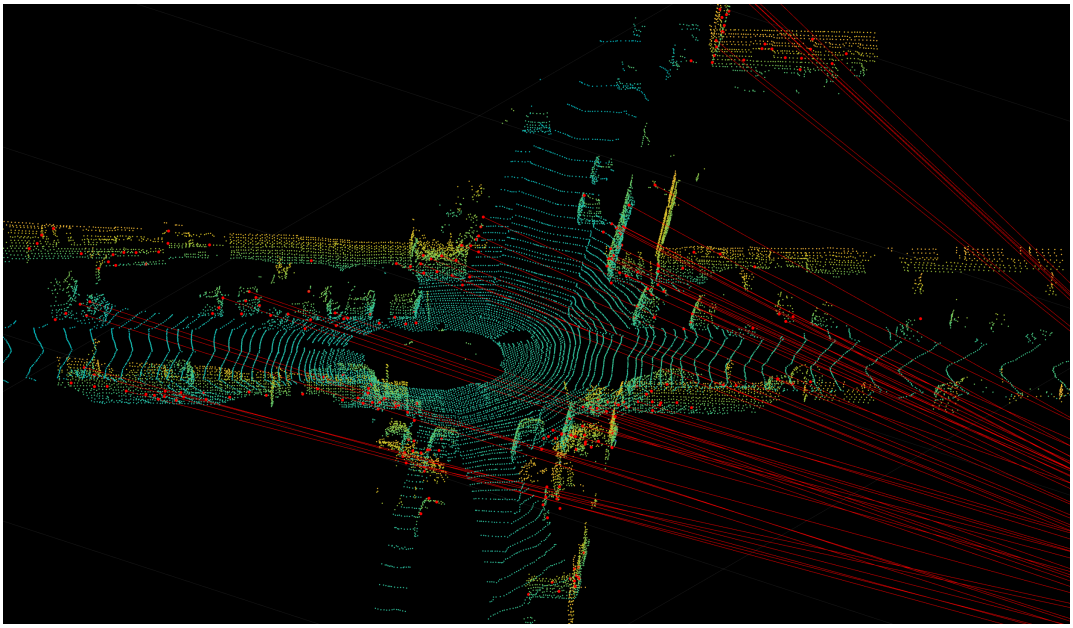

Figure 3: The first view of the intersection with the 3DFeat-Net descriptors illustrated with red dots. The lines are connecting the keypoint descriptors in their respective views to each other.

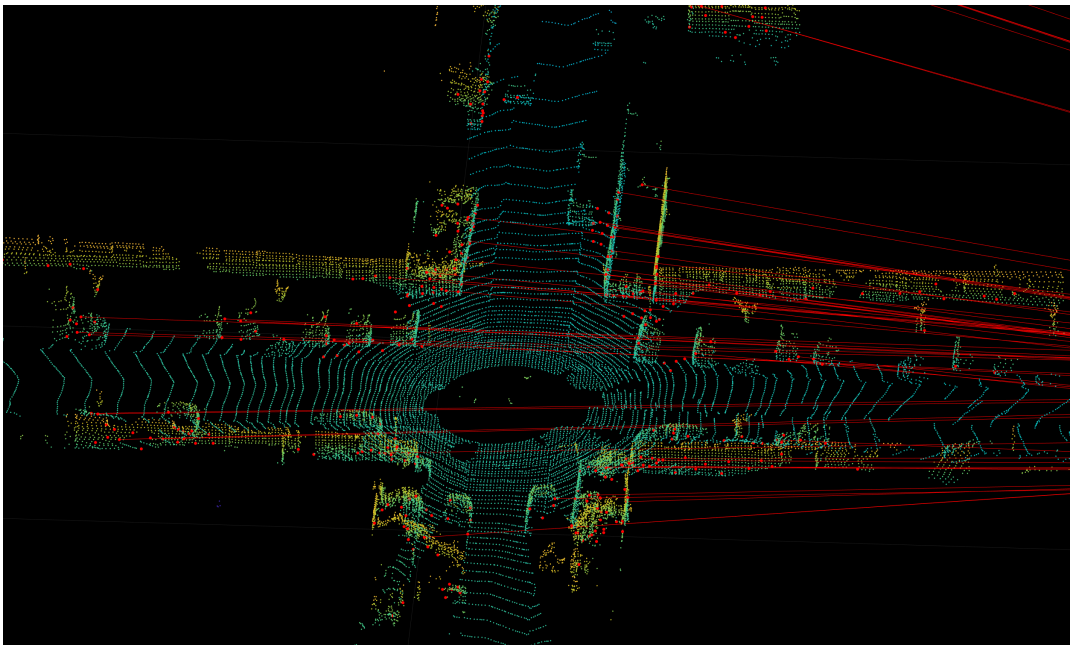

Figure 4: The second view of the intersection with the 3DFeat-Net descriptors illustrated with red dots. The lines are connecting the keypoint descriptors in their respective views to each other.

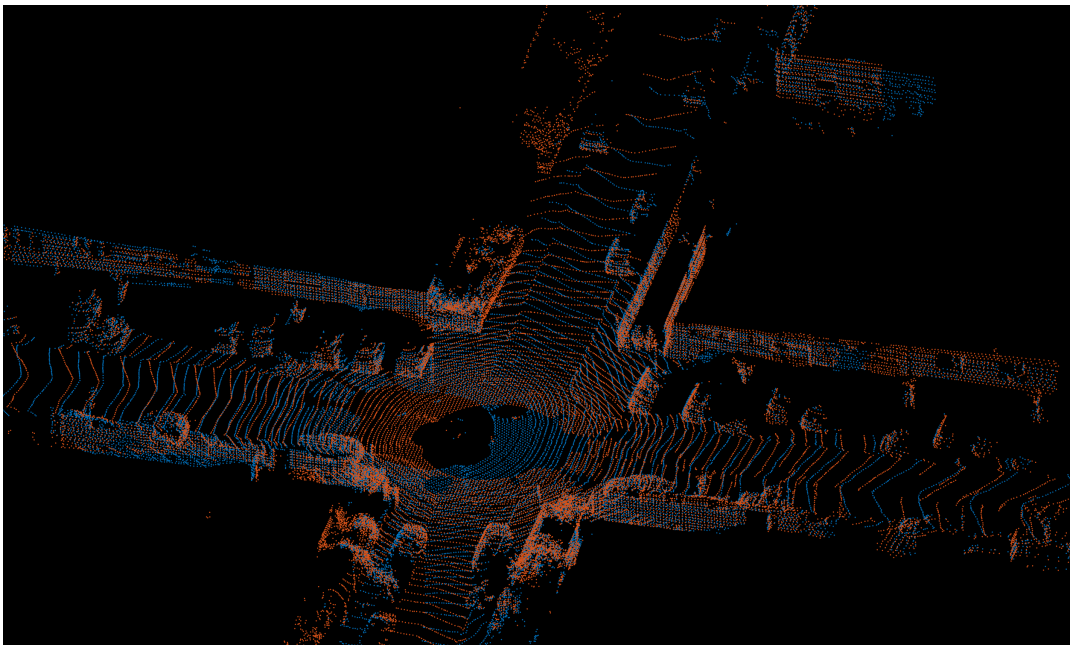

Figure 5: The aligned results of the two views. The orange LiDar points correspond to the first view and the blue LiDar points correspond to the second view.

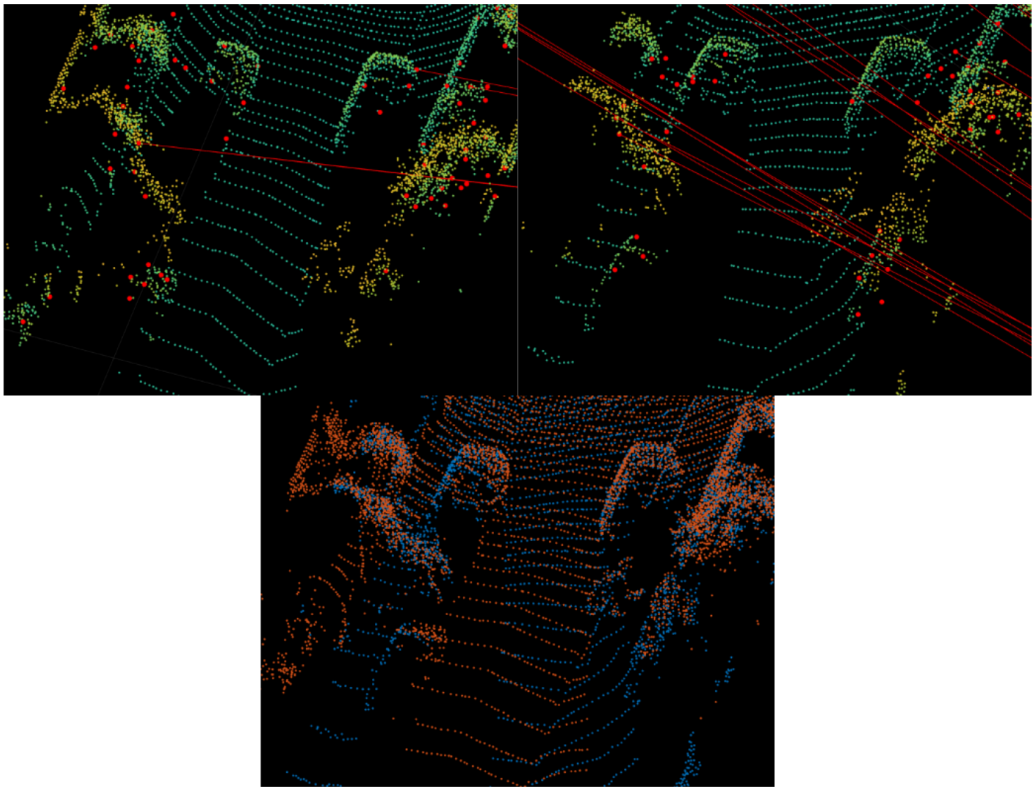

Figure 6: Viewed in view of the two views and the the stitched view to show how stitching multiple viewpoints can help make occluded objects visible.

"sight" than human drivers and will further reduce risk by letting cars better account for unexpected objects. Due to a lack of time, we were not able to collect our own dataset of the same scene from multiple viewpoints.

By relying on using KITTI on different timesteps, we were not able to get quantitative measures to show how our method would increase safety. We experimented with using VoxelNet to show how object detection would look on the two views separately and then how it would look on the stitched view. By doing this, we wanted to collect quantitative measures on how many objects the classifier could still detect as well as how many more objects that it would detect by combining the views. However, due to our method of using different timesteps of the data as different views, in our different timesteps, objects would be moving and in some stitched examples, these moving objects would not make sense to try and classify in the stitched view. Thus, for the future work, we would want these quantitative measures on some new dataset that would be collected.

5.2 Safety Implications

We believe that the safety implications of this project is immense. The stitching of the two lidar point clouds is almost instantaneous and the detection analysis on a larger lidar point cloud will roughly take the same time. The bottleneck of this algorithm is its V2V communication which has to real time and instantaneous where each viewpoint is roughly 100,000 X 4 points. This needs to be communicated to the other vehicle as the vehicles converge to the intersection.

One of the hardest applications of self-driving cars is in occluded environments and these can also be present when there is heavy fog or rain where one or more of the sensors have a high probability of producing poor results. Such V2V communication can help navigate foggy environments where camera might fail. GPS information can be used along with these lidar point clouds to produce a more accurate map of the environment. Moreover, this technique of scene stitching can be used with other sensors such as Radar on a rainy/snowy day where Lidar sensor might fail.

5.3 Future Work

The immediate future work that needs to be done is set up a evaluation metric for this project. We give two example of the quantitative metrics: analysis of detection confidence in the post and pre-stitched viewpoints and the amount of new good detections between the post and pre-stitched viewpoints as a metric to evaluate our method. Moreover, currently we use different timestamps of the same car in the intersection to create two viewpoints. Alternatively, a data set of this sort could be created to facilitate more of such projects. In the data set, an intersection can also have multiple viewpoints from different cars as well. For future work, the stitched viewpoint could also be created from multiple viewpoints from different cars.

References

[1] A. Geiger, P. Lenz, and R. Urtasun. 2012. Are we ready for autonomous driving? The KITTI vision benchmark suite. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*. 3354–3361. DOI:http://dx.doi.org/10.1109/CVPR.2012.6248074

[2] Chris Harris and Mike Stephens. 1988. A combined corner and edge detector. In *In Proc. of Fourth Alvey Vision Conference*. 147–151.

[3] Hui Chen and Bir Bhanu. 2004. 3D free-form object recognition in range images using local surface patches. In *Proceedings of the 17th International Conference on Pattern Recognition, 2004. ICPR 2004.*, Vol. 3. 136–139 Vol.3. DOI:http://dx.doi.org/10.1109/ICPR.2004.1334487

[4] A. E. Johnson and M. Hebert. 1999. Using spin images for efficient object recognition in cluttered 3D scenes. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 21, 5 (May 1999), 433–449. DOI:http://dx.doi.org/10.1109/34.765655

[5] M. Khoury, Q. Zhou, and V. Koltun. 2017. Learning Compact Geometric Features. In *2017 IEEE International Conference on Computer Vision (ICCV)*. 153–161. DOI:http://dx.doi.org/10.1109/ICCV.2017.26

[6] David G. Lowe. 2004. Distinctive Image Features from Scale-Invariant Keypoints. *Int. J. Comput. Vision* 60, 2 (Nov. 2004), 91–110. DOI:http://dx.doi.org/10.1023/B:VISI.0000029664.99615.94

[7] R. B. Rusu, N. Blodow, and M. Beetz. 2009. Fast Point Feature Histograms (FPFH) for 3D registration. In *2009 IEEE International Conference on Robotics and Automation*. 3212–3217. DOI:http://dx.doi.org/10.1109/ROBOT.2009.5152473

[8] Federico Tombari, Samuele Salti, and Luigi Di Stefano. 2010. Unique Shape Context for 3D Data Description. In *Proceedings of the ACM Workshop on 3D Object Retrieval (3DOR '10)*. ACM, New York, NY, USA, 57–62. DOI:http://dx.doi.org/10.1145/1877808.1877821

[9] Zi Jian Yew and Gim Hee Lee. 2018. 3DFeat-Net: Weakly Supervised Local 3D Features for Point Cloud Registration. *CoRR* abs/1807.09413 (2018). http://arxiv.org/abs/1807.09413

[10] A. Zeng, S. Song, M. Nießner, M. Fisher, J. Xiao, and T. Funkhouser. 2017. 3DMatch: Learning Local Geometric Descriptors from RGB-D Reconstructions. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 199–208. DOI:http://dx.doi.org/10.1109/CVPR.2017.29

[11] Y. Zhong. 2009. Intrinsic shape signatures: A shape descriptor for 3D object recognition. In *2009 IEEE 12th International Conference on Computer Vision Workshops, ICCV Workshops*. 689–696. DOI:http://dx.doi.org/10.1109/ICCVW.2009.5457637